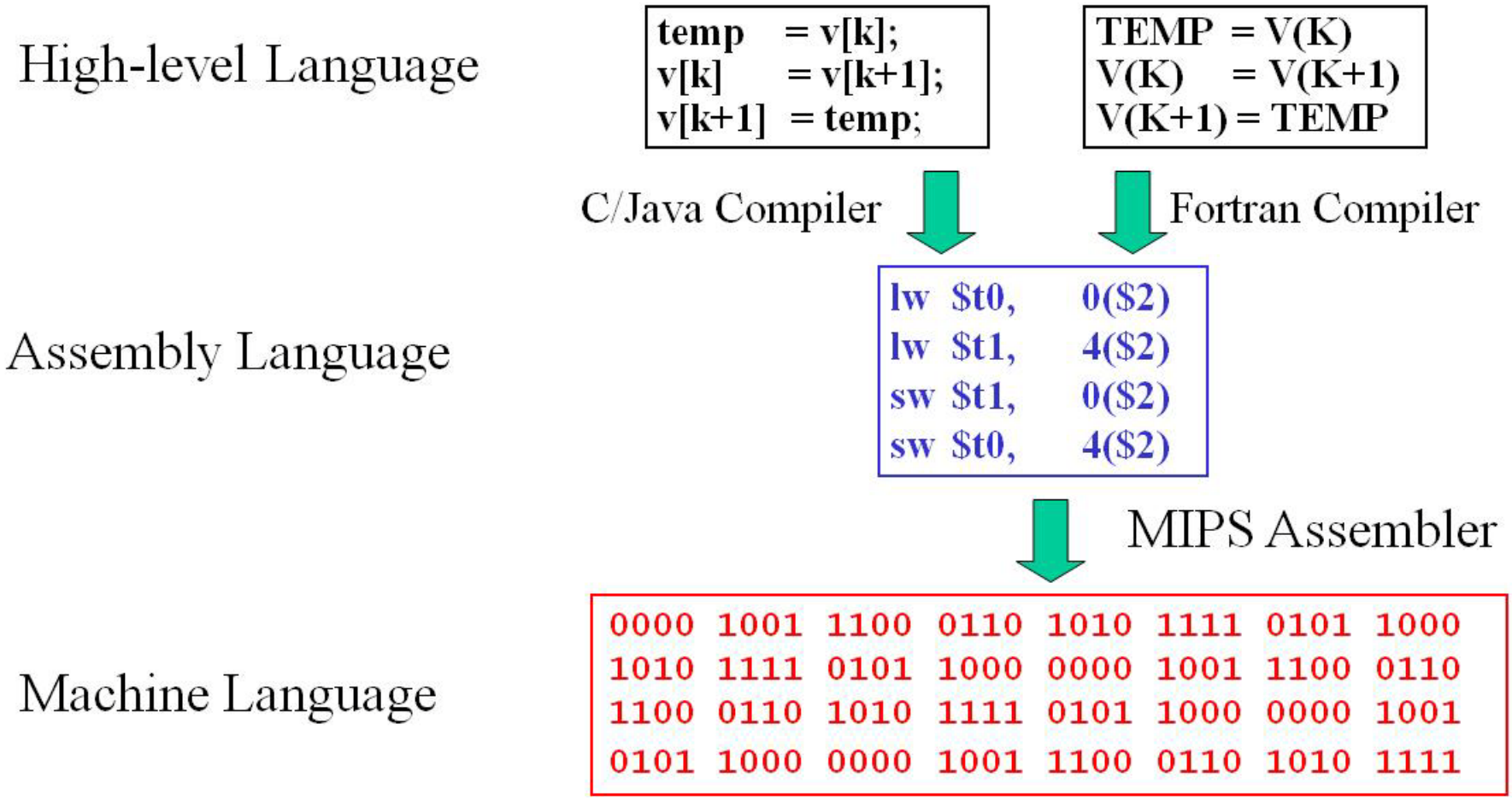
EC 220: Computer System Architecture

- **Instructor**
- Lt Col Hammad Tanveer Butt
- E-mail:
- **Credits**
- 3+1 (Lec + Lab)
- Lec : 3 hrs/wk (Meet Twice a Week 1.5 hrs)
- Lab: 3 hrs/wk (Once a week)
- Home Study: 7hrs/wk
- **Prerequisites**
- EC 210: Logic and Sequential Circuit Design
- EC 120: Computer Organization
- **Required for**
- EC 310: Microcontroller and Microprocessors

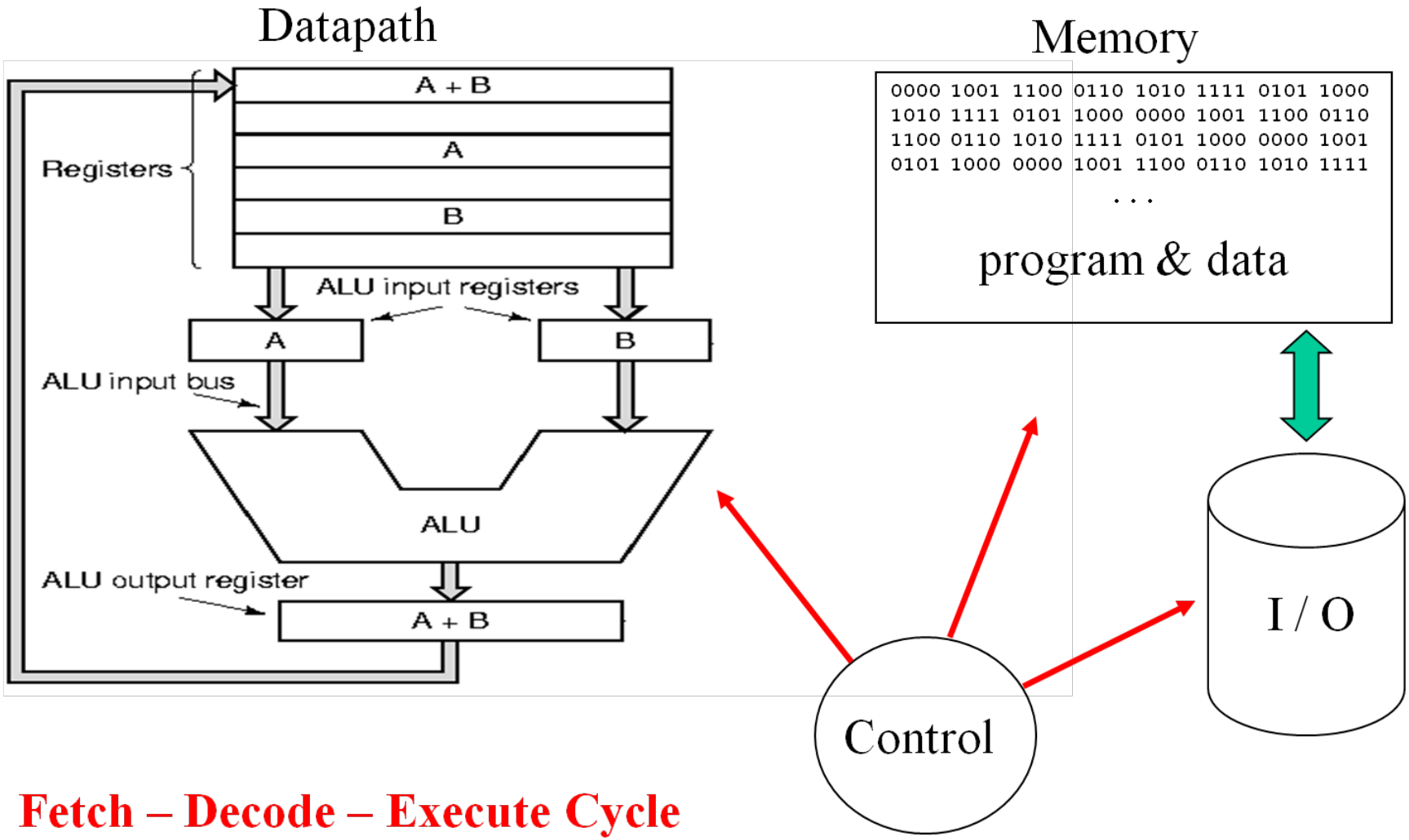
EC 220: Computer System Architecture

- *ISA: how the machine appears to a machine language programmer (or compiler)*
  - Memory model
  - Instructions
    - Formats
    - Types (arithmetic, logical, data transfer, and flow control)
    - Modes (kernel and user)
  - Operands
    - Registers
    - Data types
    - Addressing

EC 220: Computer System Architecture



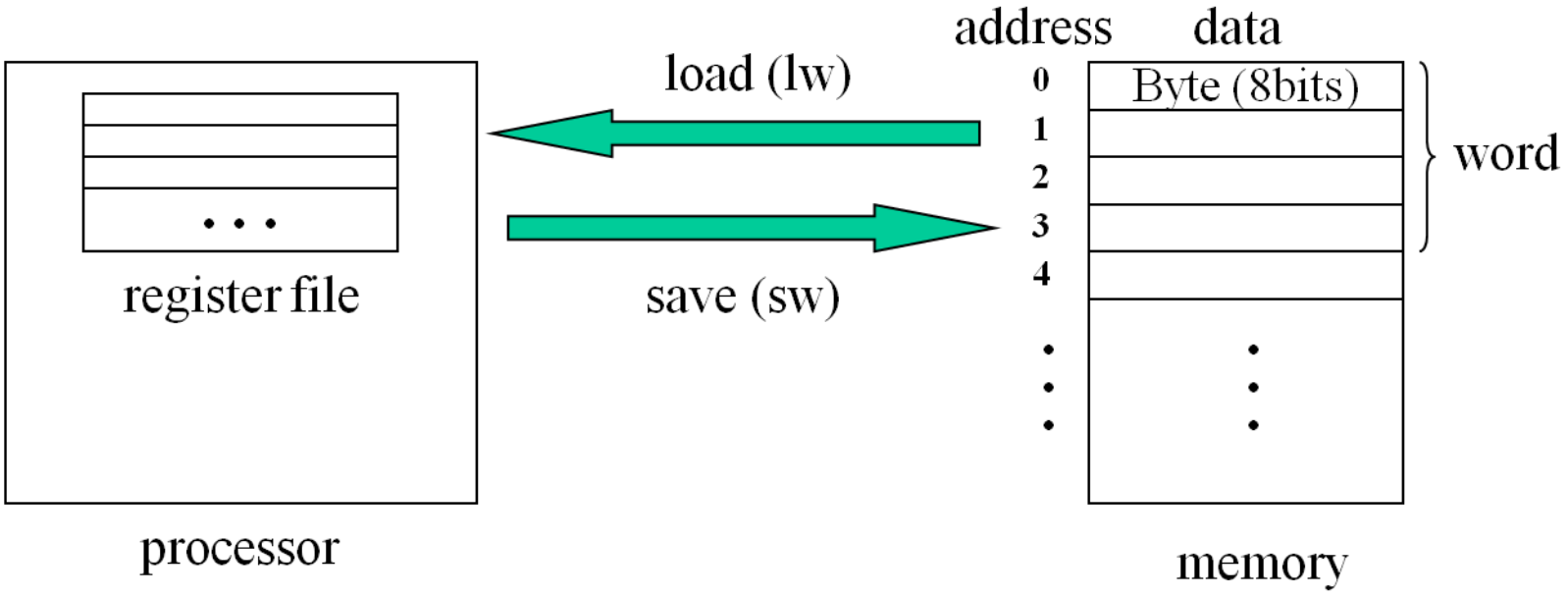
EC 220: Computer System Architecture



Fetch – Decode – Execute Cycle

Memory

- Contains instructions and data of a program
  - Instructions are fetched *automatically* by the control
  - Data has to be transferred explicitly back and forth between memory and processor
- Data transfer instructions



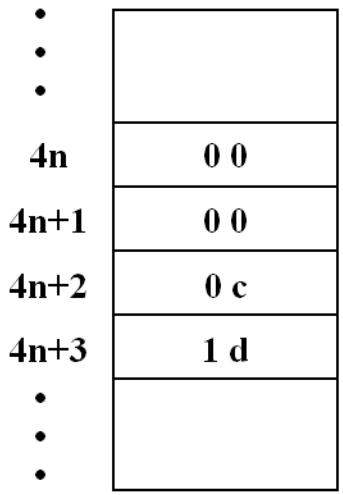
MIPS Memory Model

- Memory is byte addressable (1 byte = 8 bits)
- Only load/store can access memory data
- Unit of transfer: *word* (4 bytes)
  - M[0], M[4], M[4n], …., M[4,294,967,292]
- Words must be **aligned**
  - Words start at addresses 0, 4, … 4n
- Addresses are 32 bits long
  - 2^{32} bytes or 2^{30} words

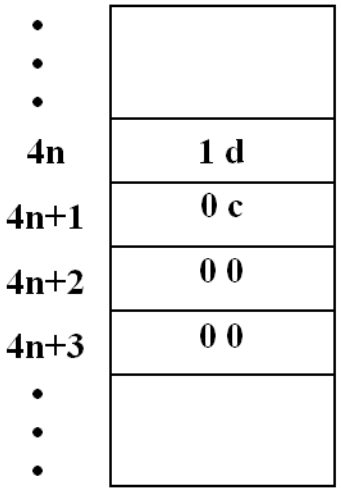
Big and Little Endian

$$(3101)_{10} = \mathbf{12} * 16^2 + \mathbf{1} * 16^1 + \mathbf{13} * 16^0$$

$$= (00\ \ 00\ 0c\ 1d)_{16}$$



big endian



little endian

MIPS Machine Language

- Arithmetic
  - **add**, **sub**, **mult**, **div**
- Logical
  - **and**, **or**, **sll** (shift left logical), **srl** (shift right logical)
- Data transfer
  - **lw** (load word), **sw** (store word)
  - **lui** (load unsigned immediate constant)
- Branches
  - *Conditional*: **beq**, **bne**, **slt**
  - *Unconditional*: **j** (jump), **jr** (jump register), **jal**

MIPS Instructions

- Simple (rigid format) instructions
  - **Add Instr**

```
add   a,   b,   c          # a = b + c;
```

Exactly 3 operands (registers) comment

  - **Given:** a = b + c + d + e ;

```
add  a,  b,  c
add  a,  a,  d
add  a,  a,  e
```

Format: <opcode> <output> <input1> <input2>

- These instructions are **symbolic** representations of what MIPS actually understands

MIPS Registers 32x 32bit

Name	Register Number	Usage	Preserved on call
$zero	0	the constant value 0	n.a.
$at	1	reserved for the assembler	n.a.
$v0-$v1	2-3	value for results and expressions	no
$a0-$a3	4-7	arguments (procedures/functions)	yes
$t0-$t7	8-15	temporaries	no
$s0-$s7	16-23	saved	yes
$t8-$t9	24-25	more temporaries	no
$k0-$k1	26-27	reserved for the operating system	n.a.
$gp	28	global pointer	yes
$sp	29	stack pointer	yes
$fp	30	frame pointer	yes
$ra	31	return address	yes

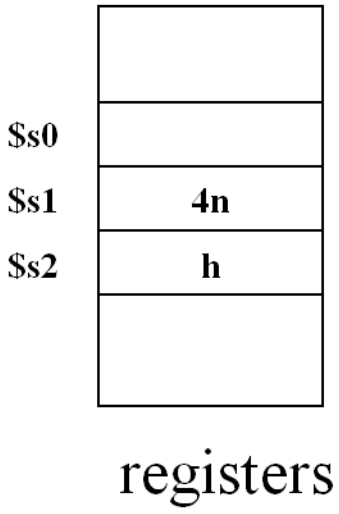
Registers vs. Memory

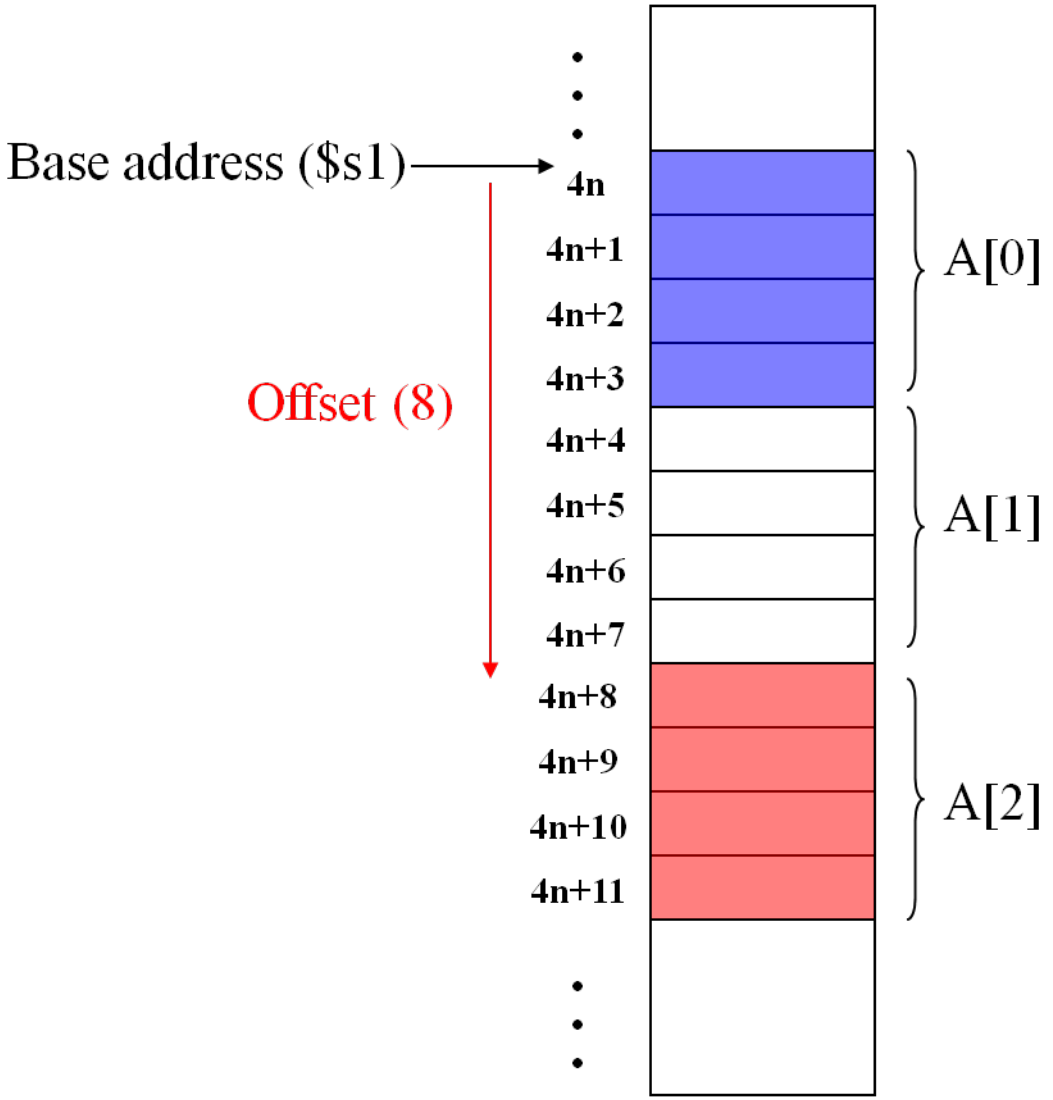
- Registers are faster to access than memory
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important!

Load, Add and Store

A[0] = h + A[2];

```
lw   $t0, 8($s1)
add  $t0, $s2, $t0
sw   $t0, 0($s1)
```



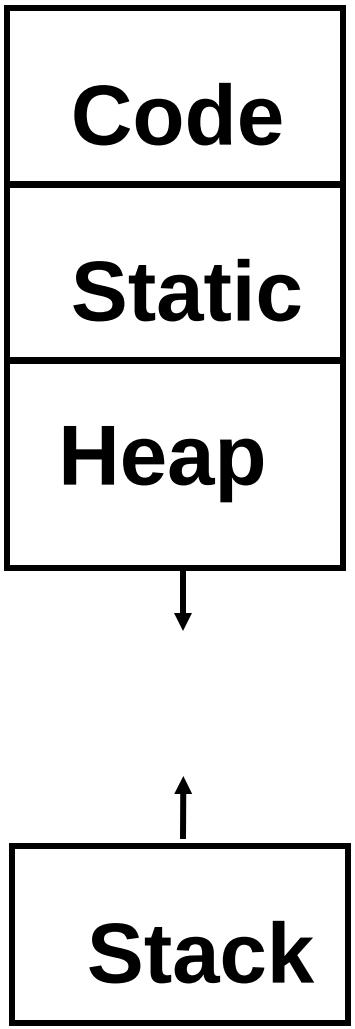


Review

- **ISA:** hardware / software interface
- **MIPS instructions**
  - Arithmetic: add $t0, $s0, $s1
  - Data transfer: lw/sw, for example: sw $t1, 8($s1)
- **Operands** must be **registers**
  - 32 32-bit registers
  - $t0 - $t7 => $8 - $15 (addresses)
  - $s0 - $s7 => $16 - $23 (addresses)
- **Memory:** large, single dimension array of bytes M[2^{32}]
  - Memory address is an index into that array of bytes
  - Aligned words: M[0], M[4], M[8], ….M[4,294,967,292]
  - Big/little endian byte order

Lifetime of Storage and Scope

- Automatic (stack allocated)
  - Typical local variables of a function
  - Created upon call, released upon return
  - Scope is the function
- Heap allocated
  - Created upon malloc, released upon free
  - Referenced via pointers
- External / static
  - Exist for entire program

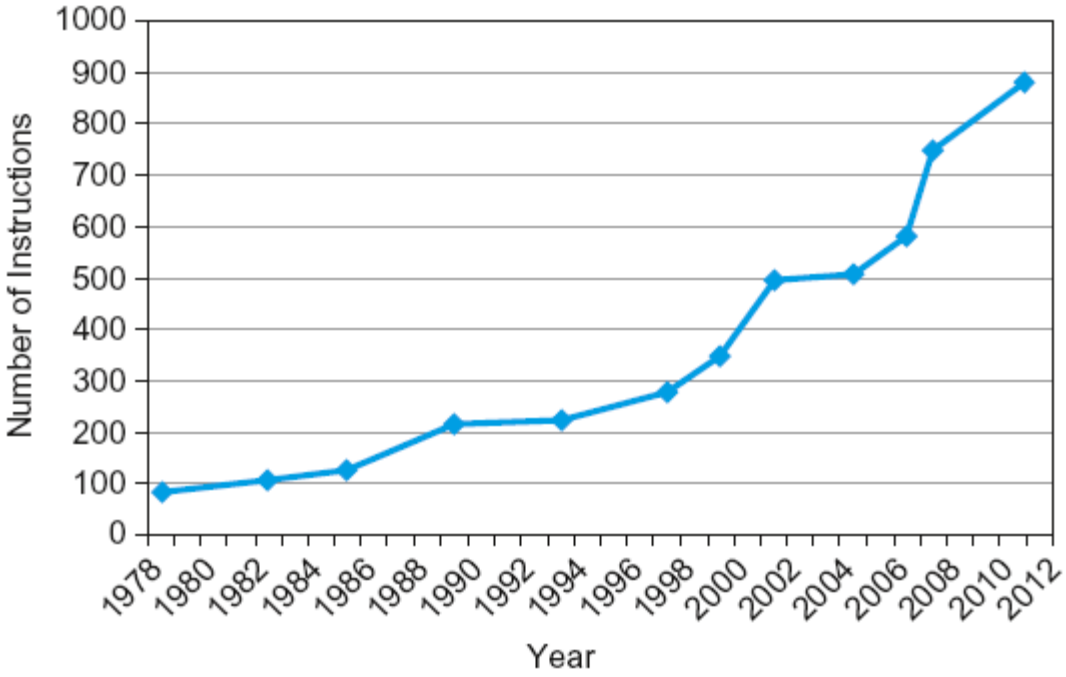


Fallacies

- Powerful instruction ⇒ higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
  - But modern compilers are better at dealing with modern processors
  - More lines of code ⇒ more errors and less productivity

Fallacies

- Backward compatibility ⇒ instruction set doesn't change
  - But they do accrete more instructions



x86 instruction set

EC 220: Computer System Architecture

EC 220: Computer System Architecture